LOGIC AND INFERENCE

CONVERTING WORDS TO NUMBERS IN PROLOG

By: Sai Koushik Neriyanuri and Gülten Sezer

OBJECTIVE

The objective of this project is to develop a Prolog program that converts numbers written in English words into their corresponding numerical values.

INTRODUCTION

In many natural language processing applications, it is useful to convert numbers written in words into their numerical form. This project focuses on developing a Prolog-based solution for this problem. Prolog, being a logic programming language, offers a unique approach to solving problems that involve pattern matching and symbolic reasoning.

LOGIC FLOW OF THE SOLUTION

The solution for converting words to numbers in Prolog follows a structured approach:

1. **Input Processing**: The input, a string of words representing a number, is split into individual word components.
2. **Word Conversion**: Each word is mapped to its corresponding numeric value using predefined base cases.
3. **Parsing Combinations**: The words are parsed to form complete numbers by combining base cases using rules that handle different numeric structures such as units, tens, hundreds, and special cases like "one thousand".
4. **Calculation**: The numeric values obtained from the words are combined appropriately to produce the final number.

MAIN COMPONENTS OF THE CODE

Base Cases:

Base cases define the numeric values for individual words representing:

Units: Numbers from zero to nine.

Teens: Numbers from ten to nineteen.

Tens: Multiples of ten from twenty to ninety.

Parsing Rules:

Parsing rules handle the combination of these base cases to interpret more complex numbers:

Hundreds: Handling phrases like "one hundred" and "one hundred and twenty three".

Tens and Units: Handling phrases like "twenty one" and "forty two".

Special Cases: Handling specific cases like "one thousand".

Helper Predicates

to_num: Main predicate that starts the conversion process.

split_string: Splits the input string into a list of words.

maplist: Converts string representations of words to atom representations.

parse_number: Core predicate that parses the list of word atoms to form the final numeric value.

CHALLENGES

During the development of this Prolog project, several challenges were encountered:

1. Handling Compound Numbers: Managing combinations of words, such as "twenty one" and "one hundred and twenty three", required careful definition of parsing rules to ensure correct interpretation and calculation.
2. Ambiguity in Input: Ensuring the program correctly parses inputs with various structures, particularly those with the conjunction "and", was a challenge. The program needed to distinguish between different numeric constructs and parse them accurately.
3. Recursive Parsing: Implementing recursive parsing rules to handle nested structures like hundreds and thousands required a clear understanding of Prolog’s recursion and pattern matching capabilities.

CONCLUSION

The Prolog project successfully converts numbers written in English words into their corresponding numerical values. By leveraging base cases and parsing rules, the program accurately processes and combines word components to form complete numbers. Despite challenges such as handling compound numbers, input ambiguities, and recursive parsing, the project demonstrates Prolog's capabilities in symbolic reasoning and pattern matching. The code is effective and serves as a strong foundation for more advanced natural language processing tasks. Future improvements could expand its scope to handle larger numbers and more complex linguistic patterns.